

# Analisi statica di Mydoom – massmail.c

Blocchi 1–4: filtro email, coda destinatari, generazione indirizzi, DNS/MX e invio SMTP

**Nota metodologica:** il documento riassume esclusivamente quanto osservato nel codice sorgente analizzato. Dove il codice non dimostra direttamente un'intenzione, viene descritta la funzionalità osservabile senza attribuire motivazioni non verificabili.

## 1. Ruolo generale del modulo

Il file **massmail.c** implementa una componente strutturata di gestione del mass mailing: normalizzazione e filtraggio degli indirizzi, gestione di una coda, generazione di ulteriori indirizzi plausibili, ricerca DNS dei record MX e preparazione dell'invio tramite moduli dedicati.



## 2. Blocco 1 – Filtro e validazione degli indirizzi email

### 2.1 cut\_email()

Riceve una stringa e tenta di estrarre un indirizzo utilizzabile. Ignora caratteri iniziali non validi, cerca la @, conserva i caratteri ammessi, converte l'output in minuscolo e rimuove caratteri finali indesiderati. Riduce inoltre anomalie come punti consecutivi nella parte del dominio.

### 2.2 email2parts()

Separa l'indirizzo in **username** e **domain**, consentendo alle funzioni successive di analizzare separatamente la parte prima e dopo la @.

### 2.3 email\_check2()

Esegue controlli sintattici: una sola @, presenza di un punto nel dominio, lunghezza della parte finale del dominio compresa tra 2 e 4 caratteri, lunghezze plausibili di username e dominio, assenza di punti consecutivi e controllo della percentuale di cifre nello username.

### 2.4 email\_filtdom()

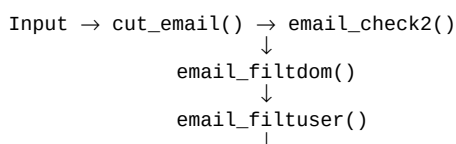
Confronta il dominio con liste interne di esclusione contenenti, tra gli altri, riferimenti a prodotti/aziende di sicurezza, domini governativi o militari e organizzazioni tecniche. È presente anche una *loyal\_list*. Il dato dimostrabile è che tali corrispondenze vengono intercettate e l'indirizzo non prosegue come destinatario valido; il motivo specifico di ogni voce non è deducibile dal solo frammento.

### 2.5 email\_filtuser()

Analizza la parte precedente alla @ ed esclude username completi come root, postmaster, webmaster, contact e altri, oltre a nomi contenenti determinate sottostringhe amministrative, tecniche o legate alla sicurezza.

### 2.6 email\_filter(): coordinamento finale

Coordina l'intera pipeline: cut\_email(), email\_check2(), email\_filtdom() ed email\_filtuser(). Gestisce anche suffissi come .nospam o .dontspam rimuovendo progressivamente l'ultima componente e ripetendo la verifica.



### 3. Blocco 2 – Gestione della coda dei destinatari

#### 3.1 massmail\_addq()

Controlla la lunghezza massima dell'indirizzo, applica `email_filter()`, evita duplicati tramite confronto case-insensitive e alloca dinamicamente una struttura `mailq_t` con `HeapAlloc()`. Inizializza stato, timestamp ottenuto con `GetTickCount()`, priorità e destinatario, quindi inserisce il nuovo elemento in **massmail\_queue**.

Gli indirizzi contenenti **.edu** ricevono un incremento del campo `priority`. Il codice dimostra l'incremento; l'effetto concreto della priorità sarà verificato nello scheduler.

```
Controllo → filtro → anti-duplicato → HeapAlloc()
          ↓
state=0 / tick_got / priority / destinatario
          ↓
        massmail_queue
```

### 4. Blocco 2 – Generazione di ulteriori indirizzi

#### 4.1 mm\_gen()

Conta gli elementi della coda, ne seleziona uno casualmente, estrae il dominio e lo combina con un nome scelto casualmente da una lista interna di nomi comuni. Il nuovo indirizzo viene reinviato a `massmail_addq()`, quindi passa nuovamente attraverso tutti i filtri.

```
paolo@azienda.com
      ↓
azienda.com + nome casuale
      ↓
john@azienda.com
      ↓
massmail_addq()
```

Il modulo quindi può creare ulteriori indirizzi plausibili sullo stesso dominio di un indirizzo già acquisito.

### 5. Blocco 3 – Cache DNS e ricerca dei record MX

#### 5.1 mmdns\_getcached()

Scorre la struttura globale `mm_dnscache` e restituisce una voce se il dominio è già presente, evitando una nuova ricerca.

#### 5.2 mmdns\_addcache()

Memorizza dominio e lista MX nella cache. La struttura contiene: collegamento al nodo successivo, puntatore alla lista MX, nome del dominio, `tick_lastused` e `ref`. La costante `MMDNS_CACHESIZE` è impostata a 256; il codice tenta di sostituire voci inutilizzate quando la cache supera la soglia prevista.

#### 5.3 mm\_get\_mx()

Prima cerca il dominio nella cache. Se non lo trova richiama **get\_mx\_list(domain)**, memorizza il risultato e restituisce la voce `cached`. Dal codice emerge una gestione esplicita dei record MX per individuare i server di posta associati al dominio destinatario.

```
Dominio → cache?
        sì → usa risultato
        no → get_mx_list()
              ↓
          add cache
              ↓
          lista MX
```

### 6. Blocco 4 – Preparazione e invio dei messaggi

## 6.1 mmsender()

Estrae il dominio dal destinatario, recupera la lista MX con `mm_get_mx()`, genera il messaggio tramite **`msg_generate(email->to)`** e passa lista MX e messaggio a **`smtp_send()`**. Il contenuto concreto dell'email non è determinabile da questa funzione: richiederà l'analisi di `msg.c`. Analogamente, il dettaglio della comunicazione SMTP sarà verificabile nel modulo che implementa `smtp_send()`.

```
Destinatario → dominio → mm_get_mx()
                    ↓
                lista server MX
                    ↓
msg_generate(destinatario)
                    ↓
                smtp_send()
```

Al termine il messaggio viene liberato con `GlobalFree()` e viene decrementato il contatore di riferimento della voce DNS utilizzata.

## 6.2 mmsender\_th(): invio tramite thread

L'invio viene eseguito in un thread separato. Il contatore globale `mmshed_run_threads` viene incrementato e decrementato atomicamente con `InterlockedIncrement()` e `InterlockedDecrement()`. Lo stato dell'elemento della coda passa da 0 a 1 durante l'elaborazione e a 2 dopo la conclusione.

```
state = 0 → non elaborato
state = 1 → invio/elaborazione in corso
state = 2 → elaborazione terminata
```

## 7. Costanti dello scheduler già individuate

| Costante              | Valore      | Significato osservabile                   |
|-----------------------|-------------|-------------------------------------------|
| MMSHED_THREADS        | 4           | Parametro relativo ai thread              |
| MMSHED_QUEUE_OVERFLOW | 4096        | Soglia critica della coda                 |
| MMSHED_UNPROC_FREEZE  | 2512        | Soglia relativa a elementi non processati |
| MMSHED_REQ_EXPIRES    | 2 × 3600 s  | Scadenza delle richieste: 2 ore           |
| MMSHED_GENTIMEOUT     | 6 × 1000 ms | Intervallo/timeout di 6 secondi           |

Il loro utilizzo concreto verrà verificato nel blocco successivo dedicato al Massmail Scheduler.

## 8. Conclusione

Nei primi quattro blocchi di `massmail.c` emerge un'architettura modulare per il mass mailing: filtraggio degli indirizzi, gestione di una coda dinamica, generazione di ulteriori indirizzi sullo stesso dominio, ricerca e caching dei record MX, generazione del messaggio e delega dell'invio al modulo SMTP. La parte successiva completerà il quadro mostrando come lo scheduler amministri la coda nel tempo, gli elementi obsoleti e i limiti di elaborazione.